HATA DEFTERİ

LEGACY defect log — 4 May 2026 build

MEB Uluslararası Robot Yarışması · Otonom Araç

Snapshot built 05 August 2026

This PDF is a read-only snapshot. The living document is `HATA_DEFTERI.md`; `PLAN_New.md` is authoritative above both.

Read section 0 before the defect list. A fault list read on its own gives a false impression of the build it describes, and section 0 is not padding.

The output of this document is section 7 — seven guards. The list of faults is only the evidence for them.

Contents

HATA DEFTERİ — the LEGACY defect log

Every fault found in the 4 May 2026 build, with evidence, cause, cost and the guard that prevents it recurring.

This is a derived document. `PLAN_New.md` is authoritative; every entry cites the plan section it came from. If the two disagree, the plan is right.

---

0. Read this before the list

A defect list read on its own gives a false impression, so this part is not padding.

What the May car actually did well

- **Bird's-eye perspective warp** before lane finding, rather than chasing pixels in a camera-angle image.
- **CLAHE on the L channel** to normalise spot reflections before thresholding.
- **Adaptive white profiles** — DARK / NORMAL / BRIGHT chosen per frame from mean brightness, instead of one fixed threshold.
- **Column-continuity weighting.** A lane line is vertically continuous in bird's-eye view; a glare spot is not. Weighting histogram columns by their vertical coverage filters reflections almost for free. This is a genuinely clever idea and it is original.
- **Near/far weighted error** — a lookahead term for curves plus a near term for position.
- **Lane memory with a narrowed search window**, which is also what already handles dashed centre lines.
- **PD control with dynamic gain, derivative capping, curve-speed coordination and dead-zone compensation.**
- **Event debouncing** — a detection must persist N frames before it fires.
- `gpiozero` **and** `picamera2`, both import-guarded for off-Pi development. Most people get the Pi 5 GPIO story wrong; this build did not.
- **A HOG sign classifier** with rotation and brightness augmentation.
- **Nine calibration tools**, including a live tuner that writes back to config.

That is a strong build. Hold onto that while reading the rest.

The actual failure mode

Read the **Cause** column in every entry below. Almost none of these are knowledge failures — nobody here failed to understand adaptive thresholding or PID control. They are **seam failures**, and nearly every seam crosses a session boundary or a tool boundary:

- The camera resolution changed in one place; the perspective quad lived in another.
- Trim grew from two variables to four; the tool that writes them stayed on two.
- A sign model was trained by one effort; its consumer was written by another; the wire between them was never run, though both sides spell the labels identically.
- Documentation was generated describing intended work, then trusted as a record of finished work.

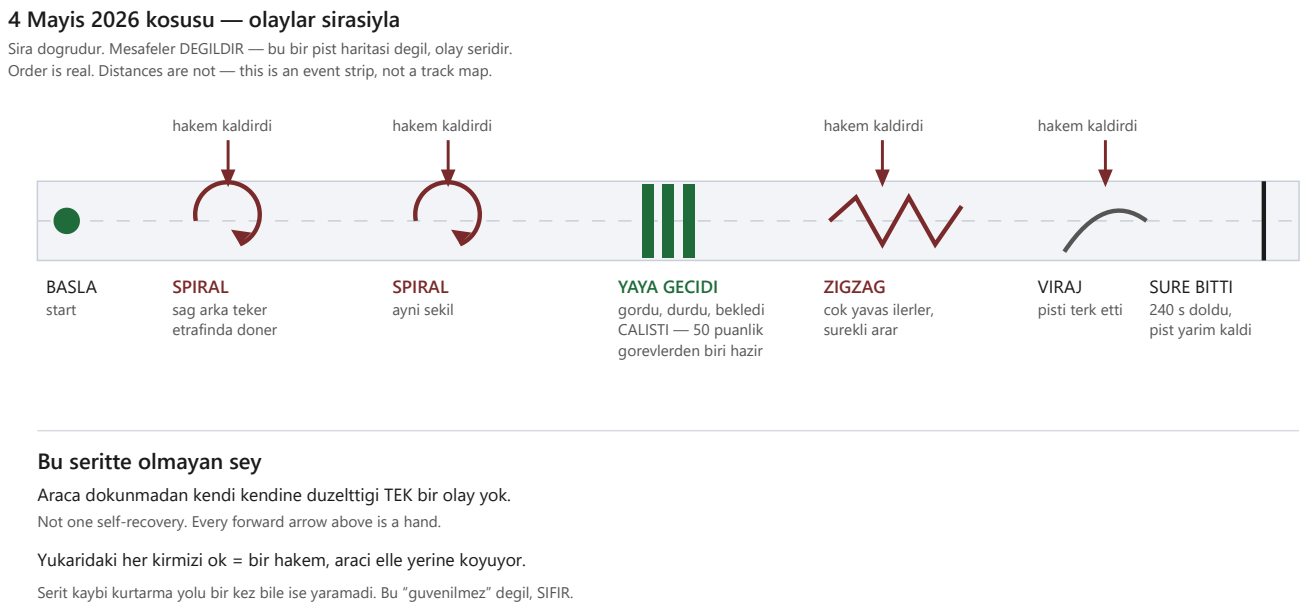
And the instrumentation was lying. Four documents dated eleven days before the competition asserted ✅ `TÜM OPTİMİZASYONLAR UYGULANMIŞTIR`, `±15 px` deviation, `28–30 FPS`, `+85–95 puan`. None of it was measured. Being wrong while your own reports say you are fine is not stupidity — it is the predictable result of a tool that confabulates and a process with no verification step.

The guards in this document exist so the seams cannot silently open again.

0.9 What the May run actually looked like

Written down 5 August 2026. Reconstructed by Egemen from competition video, as a hand-drawn diagram of the whole run.

Read the limits first. This is an external camera looking at the car, not the car's own view — it cannot be replayed through `lane.py`. The diagram is drawn from memory and footage, not measured. **No distance, angle or timing in this section is a measurement** and none may be quoted as one. What it does carry is *sequence* and *shape*, and those are the only observational data that survive from the actual competition — §3.7 records that the code which ran in Antalya was edited on site and never saved, so `LEGACY/` is not even what competed.



A note on what this drawing is and is not. It is built from Egemen's hand-drawn diagram and the video, so the *order* of events is real and the *count* of marshal interventions is real. The spacing is not — it is an event strip, not a map, and it deliberately does not claim to show the shape of the Antalya track. If the original drawing is scanned into the repo later, replace this with something geometric and delete this paragraph.

The two failure modes

The car failed in exactly two ways, each with a consistent signature.

Spiral. "*Attempts to move left, then goes right backwards, circling with the centre being the right rear tire.*"

Zigzag. "*Very slowly goes forward, unlimited searching.*"

What the spiral proves

A car rotating about one wheel means **that wheel is stationary or reversing while the other side drives forward.** There is no other way to produce it. In `controller.py` that requires `left` and `right` to have opposite signs, which reaches `_apply_dead_zone_pair`'s pivot branch — the branch that lifts both wheels to at least ± `DEAD_ZONE_MIN_PWM` independently rather than damping them.

So this is **physical evidence that opposite-sign wheel commands were being issued on track.** That could not be established from source, and nothing in the code says it should ever happen during lane following. §6 reserves pivots for the dead-end manoeuvre.

The reported order — *attempts left, then reverses right* — is a correction that changes sign and then exceeds the forward speed.

The mechanism this supports — HYPOTHESIS, not established

Defect 18 produces exactly this, and the numbers close:

1. Lane lost → `error = prev_error × 0.8`
2. The next line differentiates that fade → derivative pinned at − `DERIV_CAP`
3. `|derivative| > DERIV_SLOWDOWN_THRESHOLD` → **`speed` forced to `MIN_SPEED` = 25**
4. The same derivative drives `correction` to roughly ±80
5. `|correction| > speed` → the two wheels get opposite signs
6. Pivot branch → both lifted to at least ±30 → the car spins in place
7. It keeps spinning while the lane stays lost

One fabricated number pushes the speed *down* and the correction *up* simultaneously, which is precisely the condition for a pivot.

What would confirm it: replay track footage through `lane.py` and check whether it returns `None` on the crossing, and log `correction` against `speed` during a lost-lane episode. Both are Phase 2 work and need no car.

What the zigzag suggests

`_LOST_FRAMES_STOP` is 30 frames — a genuinely lost car should stop within about a second. It did not; the searching was described as unlimited. That means the lane was being **intermittently re-acquired**, resetting `lost_frames` before it ever reached the threshold. Found, lost, found, lost.

Why the modes appear in different places

Spirals occurred early, on the first straight. Zigzags after the crossing and around the corner. A plausible reading: the spiral needs a *large* `prev_error` to fabricate a derivative big enough to overwhelm the speed. Once the car is already crawling, that error is small, the fake derivative is small, and the result is wandering rather than pivoting. Same defect, two magnitudes.

The count that matters

Zero self-recoveries. Every advance along the track in the diagram is a marshal replacing the car by hand. Spiral → replaced. Spiral → replaced. Zigzag → replaced. Corner → replaced. The run ended on time expiry, part-way round.

The lane-lost recovery path did not succeed once in an entire competition run. That is a different statement from "unreliable", and it is countable directly off the drawing.

Also observed

- **The car drifted right most of the time.** This answers the open question in §3.2 — "*did the car pull consistently to one side?*" — with a yes, from observation rather than inference. Consistent with both untrimmed motors and the stale perspective quad; does not distinguish them.
- **The pedestrian crossing worked.** The car detected it, stopped at it, and waited. The spin happened *after* a successful task. Detection, debounce, the 30 cm threshold and the 5-second wait are functioning — one 50-point task already works.

What this does not tell us

- Not which way the car physically turned relative to its commands. `motor.py`'s `LEFT` and `RIGHT` are programmer labels; that is still question 1.
- Not whether the quad or the trims caused the drift. Only §20.7 separates those.
- No numbers. Nothing here is a measurement.

---

1. Summary

#	Defect	Category	Cost
1	Perspective quad never rescaled after resolution change	Driving	Suspected primary cause of the road-following failure
2	Motor trims never measured	Driving	Constant pull to one side
3	Balance tool writes variable names nothing reads	Tooling	Made #2 undetectable even if run
4	Trim applied twice	Latent	Squares the correction
5	Trim selected by sign, not by wheel	Latent	Correction cannot correct anything
6	`sign_type` consumed but never produced	Points	Çıkmaz yol, **100 points**, unreachable
7	Trained sign model never connected	Points	Same root as #6
8	No PWM frequency set anywhere	Driving	100 Hz default; probable cause of the dead-zone hack
9	Error handler does not stop motors	**Safety**	Up to ~1.2 s driving blind
10	Log stops at 120 s of a 240 s race	Diagnosis	Half of every run unrecorded
11	Physical start button removed	Points	**50 points** + rule exposure
12	Crosswalk uses fixed threshold while lanes adapt	Robustness	Detectors disagree under venue light
13	Two config files, one corrupted	Drift	Divergent values
14	Four fabricated documents	**Process**	The reason none of the above was caught
15	Motors driven at ~175 % of rated voltage	Hardware	Amplifies every control error
16	Two motors share one 2 A channel	Hardware	Both stall together on the bump
17	`CLAUDE.md` state list drifted from code	Drift	Phantom state, three real ones missing

2. Driving faults

1 — The perspective quad was never rescaled

Evidence. `LEGACY/config.py:23–24`

```
# ⚠️  800×680 çözünürlük için yeniden kalibre edilmeli (calibrate.py çalıştırın).
PERSP_SRC = [[160, 300], [480, 300], [0, 480], [640, 480]]
```

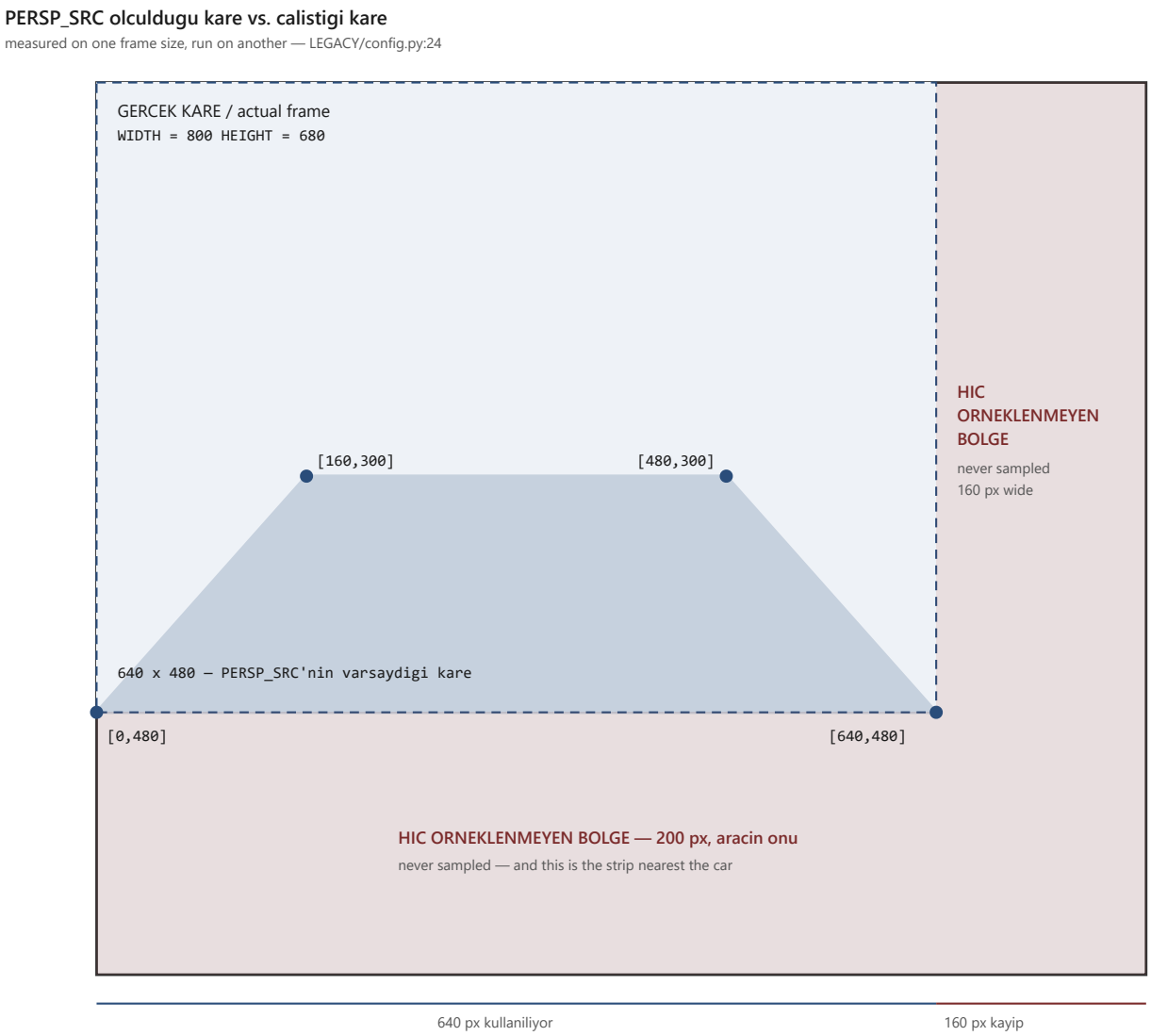
`WIDTH = 800`, `HEIGHT = 680`. Those coordinates describe a **640×480** frame.

Cause. The capture resolution was raised and every *other* resolution-dependent constant was migrated — `ROAD_ROI_BOTTOM = 680` proves the pass happened. This one was flagged with a warning and skipped, and the warning was never actioned.

Cost. The quad's horizontal centre is x=320; the frame's is x=400. `lane.py` measures error against `bird_w // 2` as though the warp were centred. It is not. A perfectly centred car reads roughly **−100 px of error**, so at `KP = 0.30` the controller applies about 30 PWM of permanent correction to a car that is exactly where it should be. **No amount of PD tuning fixes this** — the controller is faithfully correcting an error that does not exist. Separately, the quad's bottom edge sits at y=480 in a 680-row frame, so the nearest 200 rows of road never enter the warp at all.

For scale: `ASSUMED_LANE_WIDTH = 300`. The bias is about a third of a lane.

Guard. Resolution-dependent constants are derived from `WIDTH` / `HEIGHT`, or validated against them at startup and **refused** if inconsistent. A warning comment is not a guard; a failing check is. → *Plan §3.1, §20.8*



How to read it. The black rectangle is the frame the camera really delivers, 800 × 680. The dashed blue rectangle is the frame `PERSP_SRC` was measured on, 640 × 480. The blue quadrilateral is the road patch the code straightens out. The red areas are pixels that exist in every single frame and that the warp never looks at.

The bottom red strip is the worrying one: **that is the ground closest to the car**, the part that decides where it is *right now* rather than where the road goes next. The near histogram (`lane.py` step 5) is computed from the bottom of the warped image — but the bottom of the warped image is the line y = 480, not y = 680.

Nothing in the program is capable of complaining about this. `getPerspectiveTransform` accepts any four points. A quad that is too small produces a picture that looks entirely plausible. This is why defect 1's guard is a startup assertion rather than a note.

---

2 — The motor trims were never measured

Evidence. `LEGACY/config.py:101-104` — all four at `1.0`.

Cause. Step 1 of the team's own tuning workflow (`BASLA_BURADAN.txt`). Never completed, or completed and lost. See #3 for why it may have looked completed.

Cost. Cheap gearmotors are never matched. On a differential-drive car an uncorrected imbalance is a permanent pull, and the PD controller spends the run fighting a bias instead of following the road. With `KI = 0.04` the integral partly masks it on straights, then delivers the accumulated bias into the next corner.

Guard. `kontrol.py` fails if any trim is still exactly `1.0`. → *Plan §3.2*

3 — The balance tool writes names the config does not read

Evidence. `LEGACY/motor_balance_test.py:83–85` prints:

```
LEFT_TRIM  = 0.95
RIGHT_TRIM = 1.0
```

`config.py` reads `LEFT_TRIM_LOW`, `LEFT_TRIM_HIGH`, `RIGHT_TRIM_LOW`, `RIGHT_TRIM_HIGH`.

Cause. `DOSYALAR_GUNCELEME_DURUSU.txt` records it exactly: `config.py` was given speed-dependent trim profiles, while `motor_balance_test.py` was explicitly left `ORIJINAL`. Two files changed independently and were never reconciled.

Cost. This is the cruel one. The test could have been run correctly, its output pasted in exactly as instructed, and **nothing would have happened** — a dead variable sitting in `config.py` while the four live ones stayed at `1.0`. The work was possibly done and was structurally incapable of having an effect.

Guard. A tool that writes configuration **imports the names it writes**, so a rename breaks it loudly instead of silently. → *Plan §20.3e*

8 — No PWM frequency is set anywhere

Evidence. `LEGACY/motor.py:46–47`

```
self._right_pwm = PWMOutputDevice(RIGHT_PWM_PIN)
self._left_pwm  = PWMOutputDevice(LEFT_PWM_PIN)
```

No frequency argument, and no `PWM_FREQ` anywhere in `config.py`. gpiozero defaults to **100 Hz**. The earlier CascadeProjects build set `PWM_FREQ = 1000`; the setting was lost in a rewrite.

Cost. 100 Hz is coarse for motor control and probably explains `DEAD_ZONE_MIN_PWM = 30` — a minimum-power floor is exactly the workaround you reach for when low duty cycles produce no usable torque. The hack may be treating a symptom of the frequency.

Guard. Set it explicitly with a comment recording the measurement it came from. Test whether the dead zone shrinks before inheriting the hack. → *Plan §13*

---

18 — Losing the lane reverses the steering

Found 3 August 2026 by Egemen, reading `controller.py` line by line.

`compute()` handles a lost lane by fading the error out:

```
if error is None:
    self.lost_frames += 1
    error = self.prev_error * 0.8   # giderek düzleşir
```

The intent is in the docstring: "*araç yavaşlar ve son direksiyon yönünü korur*" — the car slows and keeps its last steering direction.

The next line computes the derivative from that faked value:

```
derivative = (error - self.prev_error) / dt
```

So the fade is itself read as a rate of change. Nothing on the road moved; the code invented the movement.

Worked through, with `error = +100` on the last good frame and `dt` ≈ 0.04 s:

	Last good frame	First lost frame
`error`	+100	+80 (faded)
`derivative`	≈ 0	(80 − 100) / 0.04 = **−500**, clipped to −150
P term	+39	+31
D term	≈ 0	0.45 × 1.2 × (−150) = **−81**
`correction`	**+39**	**−50**

The steering command flips sign and swings about 90 units in a single frame, and the only thing that changed is that the car went blind.

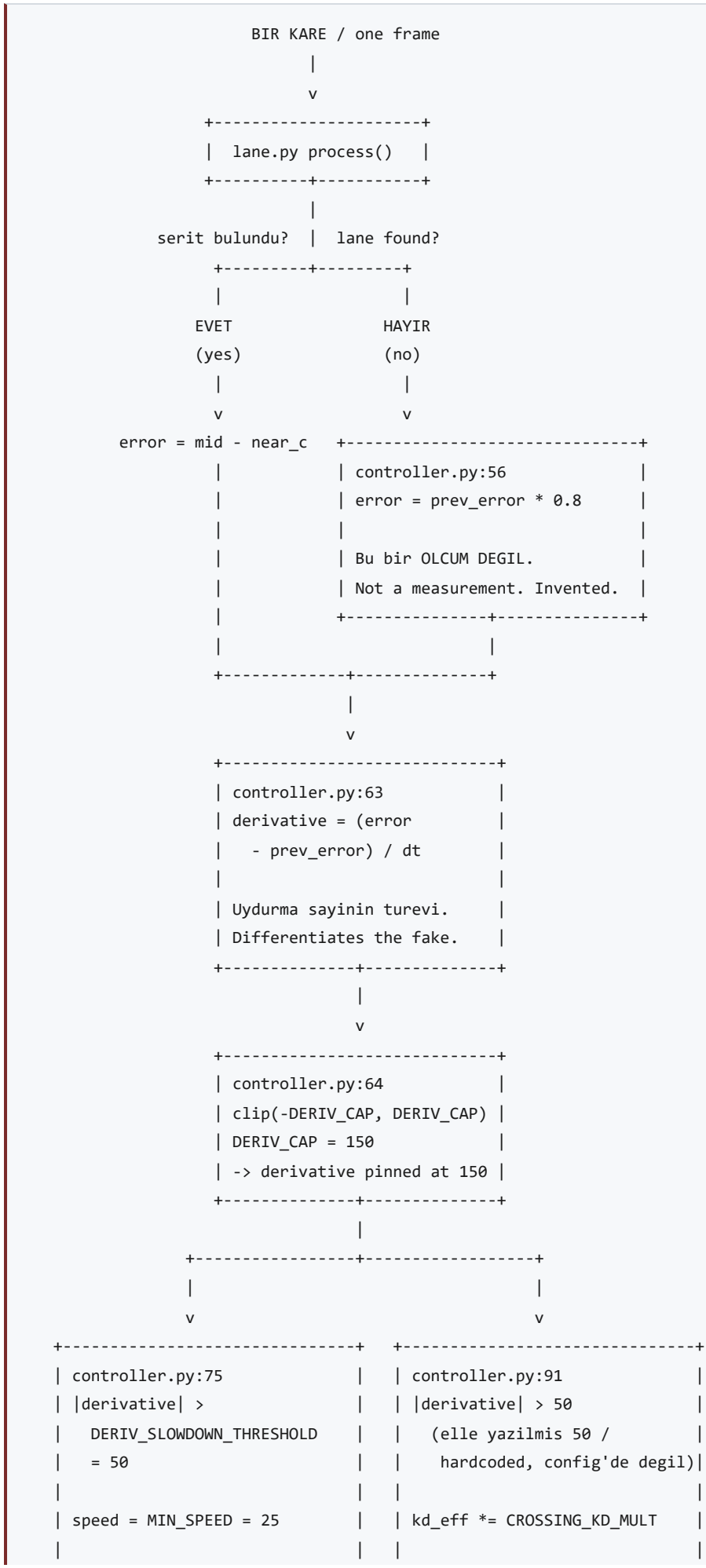
Two multipliers make it worse rather than better. `|error| > 30` raises `kp_eff` by 1.3 — but `|derivative| > 50` raises `kd_eff` by `CROSSING_KD_MULT`, because the code reads a large derivative as "we must be entering a sharp corner." It is not in a corner. It is reacting to a number the previous line made up.

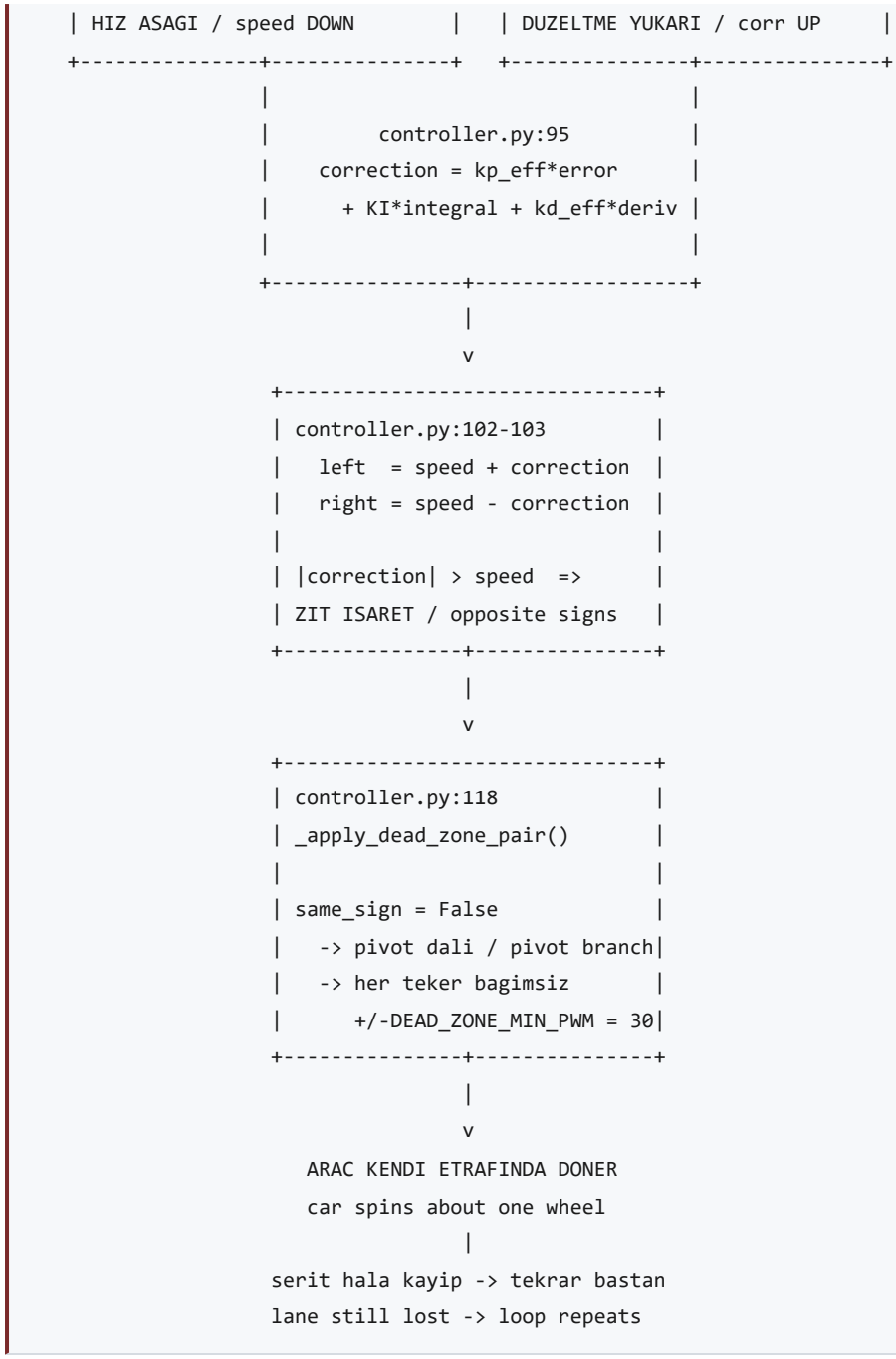
This runs in the code path that executes when the car is already lost — the reported symptom of May 2026.

Guard. When the input is synthetic, the derivative of it is meaningless. Either freeze the derivative while the lane is lost — the same way the integral is already frozen with `error_for_integration = None` — or hold the last real correction and decay *that*, rather than decaying the error and recomputing. The file already knows this pattern; it applies it to the integral and not to the derivative. → *Plan §6, §20.8*

The chain, drawn

Every box below names something `grep` can find in `LEGACY/controller.py` or `LEGACY/config.py`. Line numbers are from the files as they stand today.


BIR KARE / one frame
lane.py process()
serit bulundu?
lane found?
EVET
(yes)
HAYIR
(no)
error = mid - near_c
controller.py:56
error = prev_error * 0.8
Bu bir OLCUM DEGIL.
Not a measurement. Invented.
controller.py:63
derivative = (error
- prev_error) / dt
Uydurma sayinin turevi.
Differentiates the fake.
controller.py:64
clip(-DERIV_CAP, DERIV_CAP)
DERIV_CAP = 150
-> derivative pinned at 150
controller.py:75
|derivative| >
DERIV_SLOWDOWN_THRESHOLD
= 50
speed = MIN_SPEED = 25
controller.py:91
|derivative| > 50
(elle yazilmis 50 /
hardcoded, config'de degil)
kd_eff *= CROSSING_KD_MULT



Tek uydurulmus sayi, iki yonde birden zarar veriyor. One fabricated number pushes `speed` down and `correction` up at the same time — and a pivot is exactly the condition "correction larger than speed". They are not two separate bugs; they are one bug arriving through two doors.

Note found while drawing this: line 91 tests `abs(derivative) > 50` with the number typed directly into the file, while line 75 tests the same value through `DERIV_SLOWDOWN_THRESHOLD`. Change the constant and only one of the two thresholds moves. That is not the cause of anything observed, but it is the kind of thing that makes a later fix appear not to work.

---

19 — Speed scaling undoes the dead-zone compensation

Found 3 August 2026 by Egemen.

Three states rescale the controller's output to a fixed cruising speed:

```
elif _state == 'TUMSEK':
    l, r = controller.compute(error)
    scale = SPEED_BUMP_SPEED / max(abs(l), abs(r), 1)
    motor.set_speed(*_apply_dir(l * scale, r * scale))
```

`YAYA_YAKLAS` and `HEMZEMIN_YAKLAS` use the same three lines with `APPROACH_SPEED`.

`controller.compute()` ends by calling `_apply_dead_zone_pair`, which lifts any wheel command up to at least `DEAD_ZONE_MIN_PWM = 30` — below that the motors do not turn at all. The caller then multiplies that result by a scalar and pushes it back under the floor.

On the speed bump this is unconditional. `scale` is defined so the larger wheel lands on exactly `SPEED_BUMP_SPEED`, which is **25**. The floor is **30**. There is no pair of inputs that produces a turning wheel — `l = 200, r = 190` gives the same 25. Not a bad case; arithmetic.

On the approach states it is worse than stopping. `APPROACH_SPEED` is 35, so with `l = 70, r = 50`:

Wheel	After scaling	vs floor 30
faster	35	turns
slower	25	**stalls**

One wheel driving, one dead, while approaching a crossing the car must stop within 30 cm of. It does not slow down — it swings.

Cost. The guide allows a car stuck on the bump to be repositioned by hand and awards no points for that task. 50 points, deterministic.

Guard. Dead-zone compensation must be the **last** thing that touches a wheel command before it reaches the motor — not something a caller can undo. Either move the speed cap inside the controller so compensation is applied after it, or make `set_speed` itself the single place that enforces the floor. And any fixed speed constant must be validated against `DEAD_ZONE_MIN_PWM` at startup: `SPEED_BUMP_SPEED = 25` with a floor of 30 is a config file that cannot work, and nothing said so. → *Plan §6, §9, §20.8*

---

3. Points left on the table

6 & 7 — `sign_type` is read but never written

Evidence. `LEGACY/main.py:356`

```
sign = events.get('sign_type')
if   sign == 'cikmazsokak':  → ÇIKMAZSOKAK, brake, turn right
elif sign == 'sollamabam':   → open the no-overtaking window
```

`events.py:244–255` returns exactly `traffic_light`, `crosswalk`, `crosswalk_close`, `hemzemin`, `hemzemin_close`, `speed_bump`, `orange_car`, `yellow_car`, `parking_zone`, `sign_blue`. **There is no `sign_type` key.** `events.get('sign_type')` is always `None` and both branches are unreachable.

Meanwhile `sign_model.json` holds a trained classifier whose classes are:

```
['cikmazsokak', 'hemzemin', 'kasis', 'park', 'sollamabam', 'yayagecidi']
```

Both strings `main.py` tests for are present, spelled identically. The model was trained *for* that consumer. It is referenced only by `train_sign.py` and `sign_test.py` — never by the running system.

Cost. Çıkmaz yol is worth 100 points and could never be scored. The handling state is written correctly and is never entered. The no-overtaking window never opens either, so overtaking was permitted everywhere on the track; only the `not events['yellow_car']` check remained, which is a fallback rather than the rule.

Cause. A model built by one effort, a consumer written by another, and no step that checked the two ends met.

Guard. The event dictionary gets a **single declared schema**. A consumer reading a key no producer writes fails loudly at startup rather than returning `None` for a season. → *Plan §3.5, §20.3c*

11 — The physical start button was removed

Evidence. `LEGACY/main.py:9` — `GPIO 16 buton kaldırıldı`. Documented start paths are typing `GG` or `EZ`, or pressing `SPACE`, read through `termios` / `tty`.

Cost. The guide awards **50 points** for starting from a physical button with no external computer. A keyboard start forfeits them and requires a terminal attached to the car, which sits close to the no-external-computer rule.

Note. This is the same `GG` / `EZ` hotkey that `CLAUDE.md` names as its example of a change to reject for having no reason behind it. → *Plan §3.4*

---

4. Safety

9 — Errors are caught, motors keep running

Evidence. `LEGACY/main.py:517–525`

```
except Exception as _e:
    _err_count += 1
    print(f"[main] KARE HATASI ({_err_count}): {_e}")
    if _err_count > 30:
        _running = False
```

The loop prints and continues. **It never brakes.** Whatever PWM was last commanded stays applied for up to 30 consecutive failures — at 25 FPS, over a second of a powered car driving on stale commands.

Guard. The new loop **brakes first, then logs.** This is already the standing rule in `CLAUDE.md`: motors default off on startup and on any error, never assuming a previous safe state. The legacy code does not honour it. → *Plan §20.8*

---

5. Why none of it was noticed

10 — Telemetry stopped halfway through every race

Evidence. `LEGACY/config.py:140` — `LOG_DURATION_SEC = 120`. The race limit is **240 seconds**.

Cost. The only record the car kept switched itself off at the halfway point. A failure in the back half of the course was never going to leave evidence. Combined with rule 2.3 (radio off, so no SSH, no screen, no dashboard), the car came back with nothing to say.

Guard. `kayit.py` runs for the full race plus margin, and `kontrol.py` fails if the log duration is less than the race duration. → *Plan §21.6*

14 — Four fabricated documents

Evidence. `BASLA_BURADAN.txt`, `DOSYALAR_GUNCELEME_DURUSU.txt`, `DEGISIKLIKLER_OZET.txt`, `IMPLEMENTATION_SUMMARY.txt`, all dated 2026-04-25.

Nine constants described in detail that exist in no file — including `SHARPNESS_THRESHOLD_HIGH` and `SHARPNESS_THRESHOLD_MED`, for a sharpness feature that exists nowhere in the project in any version.

Five methods described with their behaviour that do not exist: `_get_adaptive_hsv()`, `_get_adaptive_kp()`, `_get_adaptive_kd()`, `_apply_response_mapping()`, `_apply_dead_zone_compensation()`. The last is informative — the real method is `_apply_dead_zone_pair()`, so the docs describe a **renamed predecessor**.

Ten filenames referenced that do not exist, including `FINAL_DELIVERY.md`, which `BASLA_BURADAN.txt` names as the single most important file in the project.

Wrong values: `KP` claimed `0.60`, actually `0.30`. Derivative cap claimed `±600`, actually `150`. Line counts claimed 252/205/153, actually 293/253/184.

Invented measurements: `±40 px → ±15 px`, `24–26 → 28–30 FPS` (adding per-frame CLAHE cannot *raise* frame rate), `+85–95 puan` itemised per feature.

`BASLA_BURADAN.txt` signs off with `📍 DOSYA KONUMU: /mnt/user-data/outputs/` — an AI sandbox path, copied verbatim into the repository.

Why it survived. The reports are not fiction. Six of their headline features genuinely exist — adaptive HSV, CLAHE, dead-zone compensation, speed-dependent trim, dynamic gain, derivative capping are all really in the code. **The architecture claims are true and every specific number, name, file and metric is invented.** Check two or three claims, find them correct, stop checking. A document that was wholly wrong would have been discarded in a minute.

Guard. The rule now in `CLAUDE.md`: documentation may describe only what has been read in the code or measured on a run; if a document names a constant, method or filename, that name must be findable with grep. Every fabrication above fails that test **mechanically, in about ten seconds.** → *Plan §21*

---

6. Latent, hardware and drift

4 & 5 — Two trim bugs, currently masked

Invisible while the trims are `1.0`, because multiplying by one twice changes nothing. They activate the moment real values are measured — i.e. the moment someone fixes #2.

Wrong wheel. `controller.py:165`

```
trim = LEFT_TRIM_LOW if pwm >= 0 else RIGHT_TRIM_LOW
```

Selected by the **sign of the value**, not by which wheel it belongs to. Called once for left and once for right, so whenever both drive forward **both receive the LEFT trim.** A correction applied identically to both wheels cannot correct an imbalance between them.

Applied twice. `controller.py:109–110` trims, then `motor.py:65–69` trims the same value again, that time correctly per wheel. Net: `LEFT_TRIM²` on the left, `LEFT_TRIM × RIGHT_TRIM` on the right.

`LEGACY/CLAUDE.md` records the double application as "*intentional in current code*" — a bug laundered into a design decision, which is how it survived review.

Guard. Trim lives in exactly one layer (the motor driver) and is selected by wheel identity, never by the sign of a number. → *Plan §3.3*

12 — Detectors disagree about brightness

`events.py` calls the fixed `WHITE_HSV_LOW/HIGH` for crosswalk stripes, while `lane.py` picks a DARK/NORMAL/BRIGHT profile from the scene. Under venue lighting the two can disagree about what counts as white.

Guard. One brightness decision per frame, shared by every detector. → *Plan §20.8*

13 — Two config files

`config.py` and `config_to_be_migrated.py`. The second carries the same stale `PERSP_SRC`, the same `1.0` trims, older gain multipliers (`1.0` where the live file has `1.3` / `1.2`), and its Turkish comments are **mojibake** (`TÃ¼m` — UTF-8 read as Latin-1). A corrupted older fork with nothing unique in it. → *Plan §20.3b*

15 & 16 — Hardware margins

Overvoltage. The L298N runs from 3× 18650, so 6 V motors see roughly 9–10.6 V — about **175 % of rating** — while `config.py` sets `BASE_SPEED = 62`, `MAX_SPEED = 85`. An overvolted motor is faster and more torquey than the gains were tuned for, so every correction lands harder than intended and tuning becomes much harder.

Current. Four motors, two paralleled per L298N channel, sharing one channel's ~2 A budget. Two gearmotors stalling together — exactly what a speed bump causes — will exceed it, and the overvoltage raises stall current further.

Guard. Measure at the terminals under load before trusting either. `max_pwm` starts near 57 %, not 85 %. → *Plan §3.6, §3.0*

17 — Documentation drift, the honest kind

`LEGACY/CLAUDE.md` is **broadly accurate** — its descriptions of the lane pipeline, debouncing, the logger and the gpiozero wrapper all check out. Its errors are drift, not invention: it lists a state `KIRMIZI_ISIK` that exists in no file, and omits three that do (`YAYA_YAKLAS`, `HEMZEMIN_YAKLAS`, `CIKMAZSOKAK`), so the best design in the state machine — the two-stage approach — is undocumented.

This contrast is the lesson. `CLAUDE.md` was written by *reading the code* and is reliable. The four reports in #14 were written by an assistant *describing work it believed it had done*, and are reliable about nothing. Same tool, same project, opposite trustworthiness — and the difference is only whether the author looked at the artifact. → *Plan §21.3, §21.5*

---

7. The guards, collected

Everything above reduces to seven rules. They are the actual output of this document.

1. **Resolution- and hardware-dependent constants are derived or validated at startup.** A warning comment is not a guard; a failing check is.
2. **A tool that writes configuration imports the names it writes.**
3. **Shared data structures have a declared schema.** Reading an undeclared key fails loudly.
4. **Motors brake first, then the error is logged.**
5. **Trim lives in one layer and is selected by wheel identity.**
6. **Telemetry runs for the whole race**, and a check refuses to pass if it does not.
7. **Documentation describes only what was read or measured.** Any named constant, method or file must be findable with grep, and any performance number must carry the date of the run it came from.

Rules 1, 2, 3 and 6 become assertions in `kontrol.py`, so they are enforced by a script rather than by memory. Rule 7 is in `CLAUDE.md`. Rules 4 and 5 are design constraints on the rewrite.

7b. First live test of a guard — 2 August 2026

Guard 1 (resolution-dependent constants must be validated against the live resolution) was exercised within a day of being written, and it very nearly failed.

The first mockup of the new calibration tool — the tool whose specific purpose includes preventing defect 1 — shipped with its resolution field defaulting to **800 × 600**. The camera is **800 × 680**. Had that been saved and used, it would have reproduced defect 1 exactly: a perspective quad measured against a resolution the camera does not have, and nothing in the system objecting.

Caught by comparison against `config.py` lines 11–12 before any file was written.

Two things worth taking from it:

1. **The guard is not paranoia.** A resolution mismatch is not an unlikely event that happened once in April. It is the default failure of anyone typing a plausible number from memory, and 800 × 600 is far more plausible than 800 × 680.
2. **A guard that lives only in a document does not work.** This was caught by a person reading two files side by side, not by any check. Until `ayar.py` refuses to start on a mismatch, defect 1 remains live — the entry in this notebook has changed nothing about the system's behaviour.

Recorded here rather than as defect 18 because it was caught before it reached a file. It belongs to the history of the guard, not to the list of things that shipped.

8. Closing note

Seventeen defects sounds damning until you notice that thirteen of them are seams between correctly-built parts, and that the four documents which should have caught them were instead asserting that everything had been optimised and measured.

The car was closer to working than it looked. That is worth knowing, and it is the reason §20.7 says to fix the quad, measure the trims and drive it **before** deciding how much to rewrite.